

Guide to Plotting with Seaborn and Matplotlib in Python

Data visualization is an essential part of data analysis, as it helps you understand patterns and relationships in your data. Python offers powerful libraries for creating various types of plots, including Matplotlib and Seaborn. This guide will introduce you to these libraries and show you how to create different types of plots to visualize your data.

Official Documentation:

[Matplotlib](#)

[Seaborn](#)

Table of Contents

- [Guide to Plotting with Seaborn and Matplotlib in Python](#)
 - [Table of Contents](#)
 - [Introduction](#)
 - [Installing the Libraries](#)
 - [Importing the Libraries](#)
 - [Basic Plotting with Matplotlib](#)
 - [Customizing Plots](#)
 - [Basic Plotting with Seaborn](#)
 - [Customizing Seaborn Plots](#)
 - [Common Plot Types](#)
 - [Bar Plot](#)
 - [Histogram](#)
 - [Scatter Plot](#)
 - [Box Plot](#)
 - [Heatmap](#)
 - [Advanced Customization](#)
 - [Saving Plots](#)
 - [Conclusion](#)

Introduction

Python offers powerful libraries for data visualization, with Matplotlib and Seaborn being two of the most popular. This guide will introduce you to these libraries and show you how to create various types of plots to visualize your data.

Installing the Libraries

First, make sure you have both libraries installed. You can install them using pip:

```
pip install matplotlib seaborn
```

Importing the Libraries

In your Python script or Jupyter notebook, start by importing the necessary libraries:

```
import matplotlib.pyplot as plt
import seaborn as sns
```

Basic Plotting with Matplotlib

Matplotlib is the foundation for most of Python's plotting libraries. Here's how to create a simple line plot:

```
import numpy as np

# Sample data
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Create a plot
plt.plot(x, y)

# Add a title and labels
plt.title('Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')

# Show the plot
plt.show()
```

Customizing Plots

You can customize your plots by adding more features:

```
# Create a plot with customizations
plt.plot(x, y, label='Sine Wave', color='red', linestyle='--')

# Add a title and labels
plt.title('Customized Sine Wave')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')

# Add a legend
plt.legend()
```

```
# Add gridlines
plt.grid(True)

# Show the plot
plt.show()
```

Basic Plotting with Seaborn

Seaborn is built on top of Matplotlib and provides a high-level interface for drawing attractive statistical graphics. Here's how to create a simple line plot with Seaborn:

```
# Sample data
data = sns.load_dataset("flights")

# Create a line plot
sns.lineplot(data=data, x="year", y="passengers")

# Add a title
plt.title('Number of Passengers Over Time')

# Show the plot
plt.show()
```

Customizing Seaborn Plots

Seaborn plots can also be customized:

```
# Create a line plot with customizations
sns.lineplot(data=data, x="year", y="passengers", marker='o',
color='green')

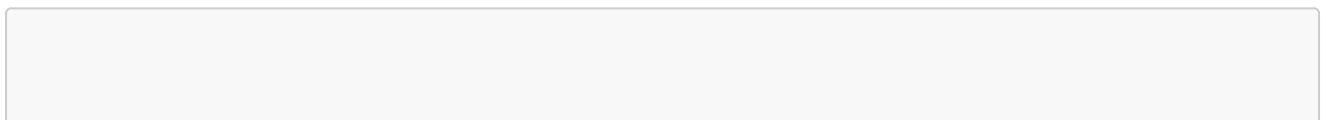
# Add a title
plt.title('Customized Number of Passengers Over Time')

# Show the plot
plt.show()
```

Common Plot Types

Bar Plot

Matplotlib:



```
# Sample data
categories = ['A', 'B', 'C']
values = [1, 4, 2]

# Create a bar plot
plt.bar(categories, values)

# Add a title and labels
plt.title('Bar Plot')
plt.xlabel('Categories')
plt.ylabel('Values')

# Show the plot
plt.show()
```

Seaborn:

```
# Sample data
data = sns.load_dataset("tips")

# Create a bar plot
sns.barplot(data=data, x="day", y="total_bill")

# Add a title
plt.title('Total Bill by Day')

# Show the plot
plt.show()
```

Histogram

Matplotlib:

```
# Sample data
data = np.random.randn(1000)

# Create a histogram
plt.hist(data, bins=30)

# Add a title
plt.title('Histogram')

# Show the plot
plt.show()
```

Seaborn:

```
# Sample data
data = sns.load_dataset("iris")

# Create a histogram
sns.histplot(data=data, x="sepal_length", bins=30)

# Add a title
plt.title('Sepal Length Distribution')

# Show the plot
plt.show()
```

Scatter Plot

Matplotlib:

```
# Sample data
x = np.random.rand(100)
y = np.random.rand(100)

# Create a scatter plot
plt.scatter(x, y)

# Add a title and labels
plt.title('Scatter Plot')
plt.xlabel('X-axis')
plt.ylabel('Y-axis')

# Show the plot
plt.show()
```

Seaborn:

```
# Sample data
data = sns.load_dataset("iris")

# Create a scatter plot
sns.scatterplot(data=data, x="sepal_length", y="sepal_width",
hue="species")

# Add a title
plt.title('Sepal Length vs Width by Species')

# Show the plot
plt.show()
```

Box Plot

Matplotlib:

```
# Sample data
data = np.random.randn(100, 4)

# Create a box plot
plt.boxplot(data)

# Add a title
plt.title('Box Plot')

# Show the plot
plt.show()
```

Seaborn:

```
# Sample data
data = sns.load_dataset("tips")

# Create a box plot
sns.boxplot(data=data, x="day", y="total_bill")

# Add a title
plt.title('Total Bill by Day')

# Show the plot
plt.show()
```

Heatmap

Seaborn:

```
# Sample data
data = sns.load_dataset("flights")

# Create a heatmap
sns.heatmap(data.pivot(index='month', columns='year',
values='passengers'))

# Add a title
plt.title('Passengers by Month and Year')

# Show the plot
plt.show()
```

Advanced Customization

Seaborn integrates well with Pandas and supports complex visualizations:

```
# Load dataset
data = sns.load_dataset("tips")

# Create a violin plot
sns.violinplot(data=data, x="day", y="total_bill", hue="sex",
split=True)

# Add a title
plt.title('Violin Plot of Total Bill by Day and Sex')

# Show the plot
plt.show()
```

Saving Plots

Both Matplotlib and Seaborn plots can be saved to files:

```
# Save a Matplotlib plot
plt.plot(x, y)
plt.savefig('sine_wave.png')

# Save a Seaborn plot
sns.lineplot(data=data, x="year", y="passengers")
plt.savefig('passengers_over_time.png')
```

Conclusion

This guide provides a basic overview of how to create and customize plots using Matplotlib and Seaborn. Both libraries offer extensive documentation and numerous options for creating complex and informative visualizations. Practice by trying out different types of plots and customizations to become proficient in data visualization with Python.